

Project Executable Files

Date	15 July 2026
Team ID	
Project Name	SkillEcho - Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes/No/NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes (requirements.txt + packages.txt)
4	Database schema / seed files (if applicable)	Yes (models.py - init_db(), seed_db())
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes (Streamlit Community Cloud)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes (functional & performance test documentation)
10	Demo video or walkthrough (if required)	NA

Step 2: File / Folder Structure

VBCUA/ .env, .gitignore, .streamlit/config.toml, packages.txt, requirements.txt, run.sh, LICENSE data/uploaded_audio/ - Student WAV submissions (UUID-prefixed) data/reference_materials/ - Admin-uploaded concept PDFs docs/1.Brainstorming & Ideation/, 2.Requirement Analysis/, 3.Project Design Phase/, 4.Project Planning Phase/, 5.Project Development Phase/, 6.Project Testing/, 7.Project Documentation/, 8.Project Demonstration/ - 22 SDLC markdown files src/frontend/app.py - Streamlit UI (~1,800 lines) src/backend/api/main.py - FastAPI REST API (795 lines, 12 endpoints) src/backend/db/models.py - SQLAlchemy ORM (464 lines, 10 tables) src/backend/services/speech.py - Whisper STT engine (95 lines) src/backend/services/nlp.py - Semantic similarity + Topic Guardrail (239 lines) src/backend/services/audio.py - Librosa DSP + filler analysis (206 lines) src/backend/services/scoring.py - Weighted evaluation matrix (180 lines) src/backend/services/reports.py - ReportLab PDF generator (414 lines)

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Deployed on Streamlit Community Cloud (live URL); FastAPI backend co-hosted via

Field	Details
	run.sh
Login Credentials (Demo)	Pre-registered demo student and admin accounts, created ahead of the live demonstration
Platform / Hosting Provider	Streamlit Community Cloud (frontend) + Neon Serverless PostgreSQL (database)
Repository Link	https://github.com/gummala-sandeep/ai-speech-evaluator
Demo Video Link	Not applicable - project was presented as a live demonstration

Step 4: Run Instructions

Pre-requisites: Python 3.11, pip, ffmpeg (system package), Git. Step 1: Clone the repository - git clone <https://github.com/gummala-sandeep/ai-speech-evaluator.git> Step 2: Create and activate the virtual environment - python3.11 -m venv venv311 && source venv311/bin/activate Step 3: Install dependencies - pip install -r requirements.txt Step 4: Configure environment - set DATABASE_URL in .env (SQLite for local dev or PostgreSQL for production) Step 5: Start both services - bash run.sh Step 6: Open the app - FastAPI backend at <http://localhost:8000> (docs at /docs), Streamlit frontend at <http://localhost:8501>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Streamlit Cloud's 1 GB RAM limit restricts Whisper to the tiny model	Module-level model caching and gc.collect() after inference keep peak RAM under 1 GB; resolved for MVP scope
2	Synchronous, single-threaded pipeline serialises concurrent evaluations	Planned Celery + Redis async task queue (Phase 2 roadmap); acceptable for current single-cohort use
3	Ephemeral disk storage - uploaded audio and PDFs are lost on instance reboot	Database records persist via Neon PostgreSQL; planned migration to S3/GCS object storage (Phase 3 roadmap)